P2500R2
C++ parallel algorithms and P2300

Published Proposal, 2023-10-15

This version:
https://wg21.link/P2500R2

Authors:
Ruslan Arutyunyan (Intel)
Alexey Kukanov (Intel)

Audience:
SG1, LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper provides the facilities to integrate [P2300R7] with C++ parallel algorithms

Table of Contents

§ 1. Motivation

C++ parallel algorithms, together with executions policies, were a good start for supporting parallelism in the C++ standard. The C++ standard execution policies represent "how" a particular algorithm should be executed; in other words, they set semantic requirements to user callable objects passed to parallel algorithms. However, there is no explicit way to specify what hardware an algorithm should be executed on.

In the absence of a better facility in the C++ standard library, custom execution policies combine semantics of both "how" and "where" the code should be executed as well a semantic restrictions on the user supplied callable. Examples can be seen in Thrust and oneDPL libraries.

[P2300R7] introduces the `scheduler` concept that represents an execution context. Compared to execution policies, `scheduler` is a more flexible abstraction for answering "where" the code should be executed, because a `scheduler` could be tightly connected to the platform it sends work to.

As [P2300R7] progresses towards likely standardization for C++26, we should answer the question how other parts of the C++ standard library would interoperate with schedulers, senders and/or receivers.

[P2214R2] outlined a plan to extend the Ranges library in C++23. This plan puts adding parallel overloads for range algorithms into "Tier 2", motivating that, among other factors, by the need to carefully consider how these algorithm overloads work when [P2300R7] lands in the standard. To the best of our knowledge, nobody has yet approached this question.

This paper is targeted to C++26 and proposes a way for standard C++ algorithms to utilize [P2300R7] facilities.

§ 2. Design overview

§ 2.1. Design goals

A key question that should be addressed is how the API of C++ algorithms, including parallel and range based algorithms, should be extended or modified to express the notion that a certain algorithm should run in a certain execution context. We strive for minimal, incremental API changes that preserve the overall usage experience as well as the core semantics of algorithms and execution policies. For example, the execution of algorithms should remain synchronous, i.e. complete all the work upon return.

However, given a wide variety of possible standard and 3rd-party execution contexts, it would be naive to expect all of them being capable of executing any C++ code. In particular, an execution context might not support certain execution policy semantics. The design should therefore allow execution semantics to be adjusted when possible, for example by using `par` instead of `par_unseq` but not vice versa.

Another important design goal is to allow implementers of a particular execution context to customize the implementation of standard algorithms for that context. We consider this a requirement so as to provide the best possible implementation for a given platform. At the same time, an algorithm should also have a default implementation, presumably expressed via other algorithms or basis routines (see § 5 Further exploration), allowing customization of only what is necessary to achieve optimal performance for a given execution context.

§ 2.2. Combining a scheduler with a policy

To achieve the first goal, we propose extending the current approach of C++ parallel algorithms to allow a *policy-aware scheduler* that combines a policy and a representation of an execution context, in addition to the existing standard execution policies. This follows the existing practice of using a single argument to specify both "where" and "how" to execute an algorithm. It forces binding a policy with a context prior to the algorithm invocation, allowing for better handling of possible mismatches between the two in case the execution context cannot properly support the semantics of the policy, as well as for reuse of the resulting policy-aware scheduler instance.

An example declaration of `std::for_each` for the outlined approach would be:

```
template <policy_aware_scheduler Scheduler, typename ForwardIterator, typer
void for_each(Scheduler&& sched, ForwardIterator first, ForwardIterator las
```

A `policy_aware_scheduler` is obtained with the `execute_on` function applied to a desired scheduler and a desired execution policy. Eventually, invoking a parallel algorithm to execute by a scheduler looks like:

```
std::for_each(std::execute_on(scheduler, std::execution::par), begin(data),
```

See § 4.3 policy_aware_scheduler and § 4.2 execute_on sections for more details.

§ 2.2.1. Why `scheduler`

The proposed API is blocking by design and behaves similarly to C++17 parallel algorithms. That means, when an algorithm returns the execution is complete. The algorithm can internally utilize a complex dependency graph of sub-computations, and a `scheduler` allows to obtain as many senders as needed to implement such a graph.

If we imagine that the algorithm takes a `sender`, it is unclear what to do then because that `sender` could represent an arbitrary dependency chain built by the user, and all possible strategies of handling it we could imagine seem bad:

- We could ignore the `sender` and just obtain the `scheduler` from it, but that is likely not what users would expect.

- We could run `sync_wait` on `sender` and then run the dependency graph that is built by the algorithm implementation, but in this case we lose the value `sync_wait` might return.
- We could build the `sender` into the algorithm implementation chain, but it is still unclear what to do with the possible return value of the `sender`. For example, it might return `int` while the algorithm semantically returns an iterator.

Furthermore, from the perspective of customization we are interested in an execution context that is exactly represented by a `scheduler`.

We believe the design does not prohibit adding other constrained functions that accepts and returns senders, should that be of interest; for example:

```
template <sender Sender, typename ForwardIterator, typename Function>
sender auto for_each(Sender s, ForwardIterator first, ForwardIterator last,
```

Such sender algorithms would not have the problems outlined in the analysis above, because of no requirement to execute immediately. The dependency graph of the computation would continue the graph to which `s` belongs, and the returned sender would signal completion of the algorithm to any receiver bound to it. In this paper, however, we do not plan to further explore this direction.

§ 2.2.2. Alternative API

An alternative API might instead take both `scheduler` and `execution_policy` as function parameters.

```
template <scheduler Scheduler, execution_policy Policy, typename ForwardIte
void for_each(Scheduler&& sched, Policy&& p, ForwardIterator first, Forwarc
```

However, in our opinion it complicates the signature for no good reason. The algorithm implementation would still first need to check if the scheduler can work with the execution policy, just on a later stage comparing to the preferred approach. Such a check would have to be redirected to the scheduler and/or the policy itself, and so would anyway require either something like § 4.2 execute_on or a member function defined by schedulers or by execution policies.

2.2.3. Extensibility considerations

In the discussion of the [P2500R1], a question about extensibility of the proposed approach was asked. In case of a need to further parameterize execution of an algorithm, for example by a memory allocation mechanism, a data partitioning strategy, a NUMA domain, etc., how would this be done?

In our opinion, these and other execution parameters conceptually map to either "how" or "where" the execution is performed - and therefore, could likely be added as configuration options for either the policy or the scheduler. For example, the best way to allocate temporary data storage needed for execution of an algorithm usually depends on which device or platform the execution should happen, and therefore should be associated with the scheduler. Data partitioning strategies, on the other hand, associate more closely with execution policies, even though best performing strategies might vary for different platforms and devices. In any case, configuring schedulers and policies seems sufficient as the extensibility approach.

That said, we are open to hear about potential configuration parameters that do not match well to a policy nor to a scheduler, and change the `policy_aware_scheduler` concept to something more generic.

2.3. Parallel algorithms are customizable functions

In line with the second design goal, we use the notion of *customizable functions* for parallel algorithms. It is essentially the same notion as proposed in Language support for customisable functions § terminology, but without specific details. Similar to the algorithm function templates in `namespace std::ranges`, these cannot be found by argument-dependent lookup. In addition, these functions can be customized for a particular policy-aware scheduler. The implementation should invoke such a customization, if exists, otherwise execute a default generic implementation. That allows customizing every particular algorithm by `scheduler` vendors, if necessary.

This paper does not explore the exact customization mechanism that might eventually be used, but it should be consistent across all algorithms which may be customized by an execution context. The practical alternatives to consider are `std::execution` § spec-func.tag_invoke and [P2547R1]. Ideally we prefer to define the parallel algorithms in a way that does not depend on a particular customization mechanism, however that might not be practical due to the syntactic differences in how customizations are declared.

2.4. Covering both "classic" and range algorithms

[P2500R0] suggested to only extend the "classic" C++17 parallel algorithms with a policy-aware scheduler, without touching the C++20 constrained algorithms over ranges. Besides being limited in scope, that also has several drawbacks:

- Keeping the existing algorithm names (`std::for_each` etc.) and yet allowing their customization requires us to:
  - Either redefine the names as customization point objects or as function objects supporting the `tag_invoke` mechanism. That would likely be considered as an ABI breaking change.
  - Or add function overloads constrained with `policy_aware_scheduler`, and require that they call new, specially defined customization point objects, like `std::for_each_cpo`. Making this for every algorithm would double the number of entities.
- The API with iterator pairs is more restrictive than with the iterator-and-sentinel pairs. One can pass two iterators as the arguments to range-based algorithms that take iterator and sentinel, while it is not possible to pass a sentinel instead of the second iterator to a "classic" algorithm.

In the current revision, we instead propose to define customizable algorithms with scheduling support in `namespace std::ranges` Implementation-wise, that most likely means extending the existing function object types with new constrained overloads of `operator()`, which we think should not create any breaking changes. The algorithm functions in `namespace std` can then be supplemented with new overloads for `policy_aware_scheduler` that are required to call respective algorithms from `std::ranges`. This approach eliminates the drawbacks described above and also addresses the desire to support the execution semantics for the range-based algorithms. The consequence is that algorithms in `std` can be customized only via range-based algorithms. We think it is a reasonable tradeoff comparing to dozens of artificial customization points or potential ABI breaks.

2.4.1. Absence of serial range-based algorithms

We understand that some range-based algorithms do not exist even as serial ones today. For example `<numeric>` does not have respective algorithms in `std::ranges`. It is supposed to be addressed either by this or by a complementary paper.

2.5. Standard execution policies for range algorithms

Since this proposal addresses the problem of extending range algorithms to work with schedulers, we think it makes sense to address the lack of execution policy overloads for range algorithms as well.

Such overloads can be safely added without any risk of conflict with the scheduler support, as an execution policy does not satisfy the requirements for a policy-aware scheduler, and vice versa.

At this point we do not, however, discuss how the appearance of schedulers may or should impact the execution rules for parallel algorithms specified in [algorithms.parallel.exec], and just assume that the same rules apply to the range algorithms with execution policies.

§ 3. Proposed API

Note that `std::ranges::for_each` and `std::for_each` are used as references. When the design is ratified, it will be applied to all parallel algorithms.

All the examples are also based on the `for_each` algorithms.

§ 4. API Overview

```
// Execution policy concept
template <typename ExecutionPolicy>
concept execution_policy = std::is_execution_policy_v<std::remove_cvref_t<E

// Policy aware scheduler
template <typename S>
concept policy_aware_scheduler = scheduler<S> && requires (S s)
{
    typename S::base_scheduler_type;
    typename S::policy_type;
    { s.get_policy() } -> execution_policy;
};

// execute_on customization point
inline namespace /* unspecified */
{
inline constexpr /* unspecified */ execute_on = /* unspecified */;
}

// std::ranges::for_each as an parallel algorithm example. Others can be do

// Policy-based API
template<execution_policy Policy, input_iterator I, sentinel_for<I> S, clas
         indirectly_unary_invocable<projected<I, Proj>> Fun>
```

```
  constexpr ranges::for_each_result<I, Fun>
    ranges::for_each(Policy&& policy, I first, S last, Fun f, Proj proj = {
template<execution_policy Policy, input_range R, class Proj = identity,
         indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
  constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
    ranges::for_each(Policy&& policy, R&& r, Fun f, Proj proj = {});

// Scheduler-based API
template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_for<I
         class Proj = identity, indirectly_unary_invocable<projected<I, Proj
  constexpr ranges::for_each_result<I, Fun>
    ranges::for_each(Scheduler sched, I first, S last, Fun f, Proj proj = {
template<policy_aware_scheduler Scheduler, input_range R, class Proj = ident
         indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
  constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
    ranges::for_each(Scheduler sched, R&& r, Fun f, Proj proj = {}) /*custo

// "Classic" parallel algorithms with scheduler
template <policy_aware_scheduler Scheduler, typename ForwardIterator, typen
  void
    for_each(Scheduler&& sched, ForwardIterator first, ForwardIterator last
```

§ 4.1. Possible implementations of a parallel algorithm

Depending on the particular customization mechanism we eventually select, a parallel algorithm can be implemented in one of the following ways.

The current design proposes that all APIs are customizable via one customization point, which is the overload that takes `I` and `S` (iterator and sentinel), and all other overloads are redirected to that customization point. We expect that implementers prefer customizing necessary algorithms just once with all associated overloads automatically covered. It is unclear at this point if there should be the flexibility of customizing every particular overload individually but we are open to exploring with interested parties where they think this might be useful.

§ 4.1.1. Customizable with `tag_invoke`

```
// std::ranges::for_each possible implementation
namespace ranges
{
namespace __detail
{
struct __for_each_fn
{
    // ...
    // Existing serial overloads
    // ...

    template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_f
             class Proj = identity, indirectly_unary_invocable<projected<I,
    constexpr for_each_result<I, Fun>
    operator()(Scheduler sched, I first, S last, Fun f, Proj proj = {}) cor
    {
        if constexpr (std::tag_invocable<__for_each_fn, Scheduler, I, S, Fu
        {
            std::tag_invoke(*this, sched, first, last, f, proj);
        }
        else
        {
            // default implementation
        }
    }

    template<policy_aware_scheduler Scheduler, input_range R, class Proj =
             indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun
    constexpr for_each_result<borrowed_iterator_t<R>, Fun>
    operator()(Scheduler sched, R&& r, Fun f, Proj proj = {}) const
    {
        return (*this)(sched, std::ranges::begin(r), std::ranges::end(r), f
    }
}; // struct for_each
} // namespace __detail
inline namespace __for_each_fn_namespace
{
```

```
inline constexpr __detail::__for_each_fn for_each;
} // __for_each_fn_namespace
} // namespace ranges
```

A customization for that approach might look like:

```
namespace cuda
{
struct scheduler
{
    template<std::input_iterator I, std::sentinel_for<I> S,
            class Proj = std::identity, std::indirectly_unary_invocable<stc
    friend constexpr std::ranges::for_each_result<I, Fun>
    tag_invoke(std::tag_t<ranges::for_each>, scheduler, I first, S last, Fu
    {
        // CUDA efficient implementation
        return std::ranges::for_each_result{last, f};
    }
};
}
```

4.1.2. Customizable with language support

Here we assume that all `std::ranges::for_each` overloads, including ones that do not take a policy or a scheduler, are defined as `customizable` or `final` functions (in the sense of [P2547R1]). We have not explored if it is practical to change the existing implementations of range algorithms in such a way.

```
// std::ranges::for_each possible implementation
namespace ranges
{
    // ...
    // Existing serial overloads
    // ...

    template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_f
             class Proj = identity, indirectly_unary_invocable<projected<I,
    constexpr for_each_result<I, Fun>
```

```
    for_each(Scheduler sched, I first, S last, Fun f, Proj proj = {}) custo

    template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_f
             class Proj = identity, indirectly_unary_invocable<projected<I,
    constexpr for_each_result<I, Fun>
    for_each(Scheduler&& sched, I first, S last, Fun f, Proj proj = {}) def
    {
        // default implementation
    }

    template<policy_aware_scheduler Scheduler, input_range R, class Proj =
             indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun
    constexpr for_each_result<borrowed_iterator_t<R>, Fun>
    for_each(Scheduler sched, R&& r, Fun f, Proj proj = {})
    {
        return std::ranges::for_each(sched, std::ranges::begin(r), std::ran
    }
}
```

§ 4.2. `execute_on`

`execute_on` is the customization point that serves the purpose of tying a `scheduler` and an `execution_policy`.

A possible implementation is:

```
namespace __detail
{
class __policy_aware_scheduler_adaptor; // exposition-only

struct __execute_on_fn {
    template <typename Scheduler, execution_policy Policy>
    auto operator()(Scheduler sched, Policy policy) const
    {
        if constexpr (std::tag_invocable<__execute_on_fn, Scheduler, Policy
        {
            return std::tag_invoke(*this, sched, policy);
        }
        else
        {
```

```
            return __policy_aware_scheduler_adaptor(sched, std::execution::
        }
    }
}; // __execute_on_fn
} // namespace __detail

inline namespace __execute_on_fn_namespace
{
inline constexpr __detail::__execute_on_fn execute_on;
} // __execute_on_fn_namespace
```

Unless specially restricted, scheduler types which will be defined in the standard should be capable and therefore required to support all the standard execution policies. For these schedulers, falling back to sequential execution for any unknown 3rd party policy may also be desirable.

Generally, however, it is up to the scheduler to decide whether to provide a fallback for unknown and unsupported policies or not, and if yes, to define what this fallback is doing. For example, SYCL-based or CUDA-based schedulers cannot fallback to sequential execution because it is generally impossible. Such a scheduler might represent an accelerator where sequential execution is not supported. Executing sequentially on a CPU might be incorrect too if the data is allocated on another device and inaccessible for the CPU.

Based on the above, we propose that:

- `execute_on` has the default implementation that somehow matches the given scheduler with the `sequenced_policy`;
- if for the given `scheduler` and execution policy the customization of `execute_on` fails to compile or causes a runtime error, it is still a valid implementation.

A customization might look like:

```
// tag_invoke based customization
namespace sycl
{
    template <typename ExecutionPolicy>
    constexpr auto is_par_unseq_v; // exposition-only

    class __policy_aware_scheduler_adaptor; // exposition-only

    struct scheduler {
        template <std::execution_policy Policy>
```

```
        friend auto tag_invoke(std::tag_t<std::execute_on>, scheduler sched
        {
            static_assert(is_par_unseq_v<Policy>, "SYCL scheduler currently
            return __policy_aware_scheduler_adaptor{sched, std::forward<Pol
        }
    };
}
```

§ 4.3. `policy_aware_scheduler`

`policy_aware_scheduler` is a concept that represents an entity that combines `scheduler` and `execution_policy`. It allows to get both execution policy type and execution policy object from the `policy_aware_scheduler` returned by `execute_on` call.

> NOTE: `policy_type` and `execution_policy` object are not necessarily the same which `execute_on` was called with.

```
template <typename S>
concept policy_aware_scheduler = scheduler<S> && requires (S s) {
    typename S::base_scheduler_type;
    typename S::policy_type;
    { s.get_policy() } -> execution_policy;
};
```

See § 4.4 execution_policy concept for more details about `execution_policy` concept.

Customizations of the parallel algorithms can reuse the existing implementation (e.g., TBB-based, SYCL-based, CUDA-based) of parallel algorithms with `ExecutionPolicy` template parameter for "known" `base_scheduler_type` type.

§ 4.4. `execution_policy` concept

The execution policy concept is necessary if we want to constrain the return type of the `s.get_policy()` method for `policy_aware_scheduler`.

Since the scheduler tells "where" algorithms are executed and policies tell "how" algorithms are executed, we consider the set of policies currently defined in the `std::execution` namespace to be

sufficient. So, the concept definition could look like:

```
template <typename ExecutionPolicy>
concept execution_policy = std::is_execution_policy_v<std::remove_cvref_t<E
```

We are open to make it more generic to allow adding custom policies for a particular scheduler, if somebody sees the value in it. For that case we either need to allow specializing `std::is_execution_policy` or to define another trait.

5. Further exploration

The authors plan to explore how to specify a set of basic functions (a so-called "parallel backend") which parallel algorithms can be expressed with. It might be proposed in a separate paper based on the analysis.

6. Revision History

6.1. R1 => R2

- Changed `execute_on` implementation to advice sequential execution
- Allowed `execute_on` customizations to cause a compile-time or a run-time error
- Added customization examples for an algorithm and `execute_on`
- Added a subsection on extensibility of the proposed approach
- Added a possible signature of a lazy `for_each` algorithm

6.2. R0 => R1

- Defined the API in terms of "customizable functions" instead of CPO
- Set range-based algorithms as the primary customization point for schedulers
- Proposed support for standard execution policies to range-based algorithms
- Defined scheduler-aware parallel algorithms in `namespace std` via constrained overloads redirecting to the range-based analogues
- Clarified behavior of execute_on

§ 7. Acknowledgements

- Thanks to Thomas Rodgers for reviewing and improving the wording, and for his feedback on execution semantics.

§ References

§ Normative References

[P2300R7]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 21 April 2023. URL: https://wg21.link/p2300r7

[P2547R1]

Lewis Baker, Corentin Jabot, Gašper Ažman. *Language support for customisable functions*. 16 July 2022. URL: https://wg21.link/p2547r1

§ Informative References

[P2214R2]

Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 18 February 2022. URL: https://wg21.link/p2214r2

[P2500R0]

Ruslan Arutyunyan. *C++17 parallel algorithms and P2300*. 15 October 2022. URL: https://wg21.link/p2500r0

[P2500R1]

Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel algorithms and P2300*. 17 May 2023. URL: https://wg21.link/p2500r1